Week 6 – Design Studio: Answers

Design the airline's disruption assistant

> Read this after the studio. It isn't the only good answer; it's what a strong answer contains. The system is the retrieval-grounded assistant from the lecture, with a passenger and a flight in place of an employee and a policy, and the differences are in the rates: three sources that change monthly, hourly and by the minute, a question peak that arrives in the same hour the documents move fastest, and an answer that commits the airline to money.

The numbers, as given

The prompt supplies them; what a strong answer adds is the implication of each.

NUMBER	VALUE	IMPLICATION
questions at the peak	150 a second, in the storm hour	the cache carries most of the load; the vendor's 50 a second is the bottleneck, not the index; the index itself is small and in memory
the answer	the waiver quoted, within 5 s	one generation per answer, streamed; a money claim is a verbatim quote, so a check in code can verify it
the fare rules	2,000 documents, monthly	indexed on the schedule; nothing about freshness turns on them
the waivers	5 to 40 a day, each with routes, dates and an expiry; changed within the hour	indexed on the change, in minutes, not on the nightly schedule; the expiry rides with every chunk and is checked again at the answer
the flight status	1,500 flights, several changes an hour each	never indexed; a table overwritten every minute and read by flight key at the request
the vendor	50 generations a second; 300 ms to embed; 2 to 4 s to generate	at the peak, two in three questions cannot be generated; the cache and the degraded mode are not optional
unanswerable	about a third	the floor, the handoff and the unanswerable slice of the labelled set are a third of the system's work, not an edge case

1. Architecture

Before the request, every few minutes, and at the request

Monthly, when a fare rule changes. The policy team publishes a new version through review. The change feed carries it to the nightly index build: fetched, chunked by clause with the heading path, embedded with the pinned model, inserted into both indexes while the old version's chunks are deleted in the same run; the check runs; the pointer moves. Nobody is waiting.

Within minutes, when a waiver is published, extended or withdrawn. The same index build, started by the waiver feed instead of the clock. A waiver is one chunk: short, and its condition (which routes, which dates, until when) travels with its rule. Its expiry is copied onto the chunk as metadata. A withdrawn or expired waiver's chunk is deleted in the run that learns of it. The design's freshness number for waivers is the time from publish to pointer, and it is measured: minutes.

Every five minutes, the briefing. One inference over the active waivers and the status feed, with its own instructions: which airports are affected, which waivers are active and until when, what rebooking capacity exists tonight. A check in code: every line cites a waiver id or the feed, the whole is under a few

hundred tokens, and it is dated. Published with a version and an as-of; a failed check leaves the previous briefing in place. The briefing is in every prompt until the next run.

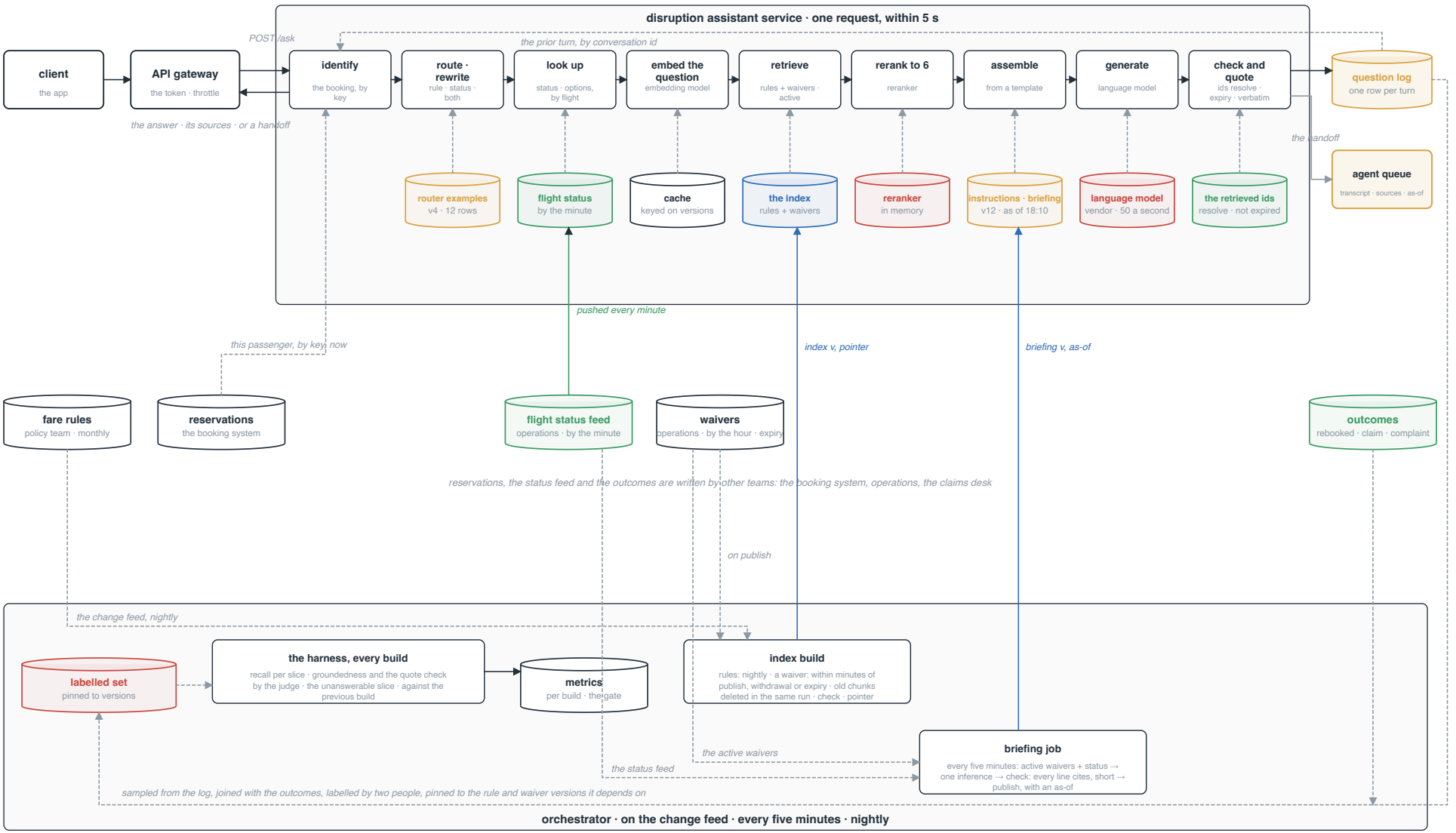
Every minute, the flight status. Operations pushes every flight's status, gate, new departure time and the rebooking options its system has already found into a table the service holds in memory. Overwritten, never appended, never embedded. The assistant reads it by flight key.

At the request. Identify the passenger and their booking; route the question (a rule, a status, or both) and rewrite a follow-up into a standalone question; look up the flight; embed the question; retrieve from the index, filtered to active waivers on this route and these dates; rerank to six; assemble the context from a template; generate; check every citation and every quote; write the log; answer, or hand off.

The seam. The index, the briefing, the status table and the router's examples, each with a version or an as-of; the request path reads them and never writes them. The booking is read from the reservation system by key and is nobody else's.

The architecture

The components: what is one service, what is orchestrated, where the data flows. Solid arrows are the request path and writes; dotted arrows are reads; blue is an index or briefing version arriving in the service.



The disruption assistant: the service, the sources at their three rates, the orchestrator, and the flow between them. Solid: the request path and writes. Dotted: reads. Blue: the index and the briefing, versioned.

Reading it: the assistant service runs the whole request path in one process, with the router's examples, the status table, the cache, the index, the reranker, the instructions and the current briefing in memory, and the vendor's models behind their APIs. The identify stage reads the booking from the reservation system by key, and the prior turn from the question log by conversation id. The status feed pushes into the status table every minute. The index build runs nightly for the rules and within minutes of a waiver's publish, withdrawal or expiry; it deletes the old chunks in the same run it inserts the new, checks, and moves the pointer the service reads. The briefing job runs every five minutes over the active waivers and the status feed. The last stage writes every answer to the question log, whole, with every as-of it was built from, and hands off to the agent queue when it does not answer. The harness runs on every build against a labelled set sampled from the log, joined with the outcomes and pinned to the versions it depends on, and writes the metrics the gate reads.

Stage by stage, with the artifact each stage reads and how old it may be:

STAGE	WHAT IT DOES	CODE OR MODEL
identify	the passenger from the token; their booking, flight key, fare class and what they have already been offered, from the reservation system by key; the prior turn by conversation id	two reads; no model
route · rewrite	a label, RULE, STATUS or BOTH, from a dozen worked examples in the prompt; for a follow-up, the question rewritten from the prior turn into one that stands alone	one inference, a few tokens out, parsed by code; the examples are a versioned file with a slice in the harness
look up	the flight's status, gate, new time and the rebooking options already found, from the status table by flight key	a table read; a minute old at most, and the minute is recorded
embed the question	the rewritten question, with the pinned model	a model, 300 ms; skipped on a cache hit
retrieve	lexical and vector candidates from the index, filtered inside the search to the rules and to waivers that are active, cover this route and these dates; fused	the index, versioned; the filter is a property of the chunk's metadata
rerank to 6	the question against each candidate, kept to six	a model, pinned
assemble	from a template: the instructions, the examples' labels, the briefing with its as-of, the lookups with their minute, the six passages with ids and expiries, the prior turn, the question	code; about 4,000 tokens against a budget
generate	one inference, streamed; the draft cites every fact as [n] and quotes verbatim any sentence that commits the airline to money, with the waiver id	the language model, pinned
check and quote	every [n] is a retrieved id; every quoted span is a substring of its passage; every quoted waiver's expiry is later than now, read from the chunk's metadata; the draft is not NOT_FOUND; within length. A draft that fails is not shown	code, on every answer; the judge samples behind it

The cache

Ten thousand passengers on the same cancelled flight ask the same thing. The key is the rewritten question, normalized, together with the index version, the briefing version and the status minute for this flight key. The value is the finished answer with its quotes and ids. An entry is wrong the moment any part of the key moves: a waiver withdrawn (new index version), a briefing run (new briefing version), a gate change (new status minute). Nothing is invalidated by hand; the key carries the versions, so a stale entry is simply never looked up again. The booking is not in the key, because the answer is written for the flight, and the identify stage adds the passenger's own lines (what they were already offered) outside the

cached text. The hit rate during a storm is the number that makes 150 a second possible on 50 generations a second.

The briefing

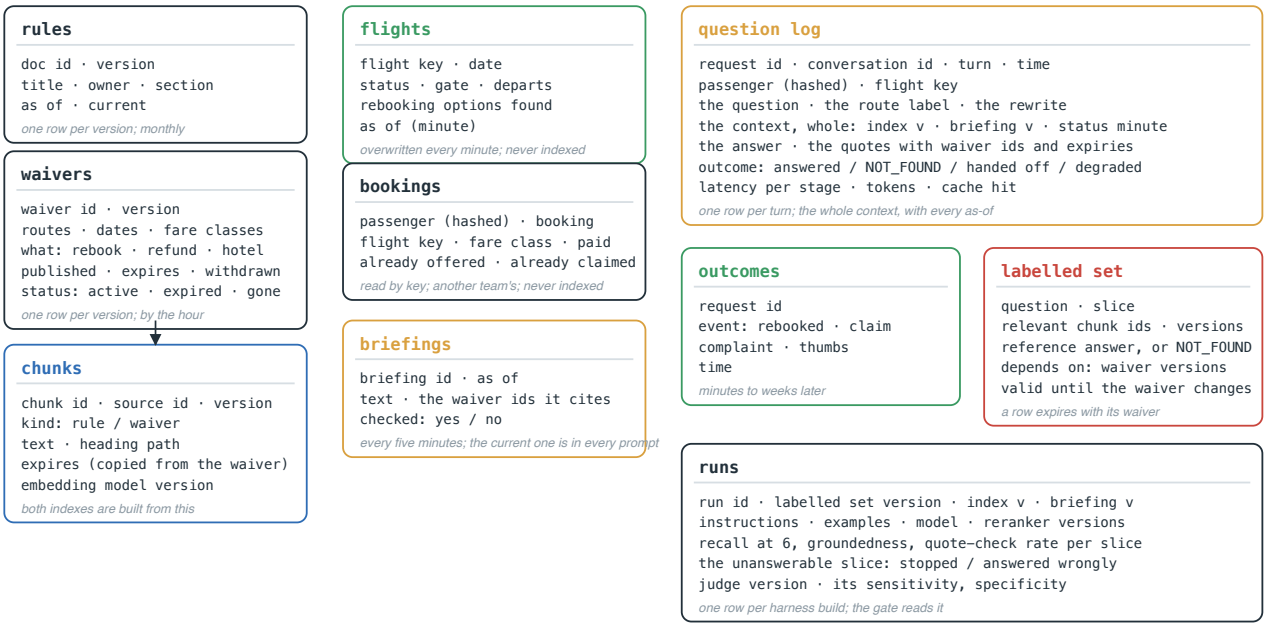
What it says: the airports affected, the waivers active with their ids and expiries, the rebooking capacity tonight, the hotel situation at the hub. Who builds it: a job, every five minutes, with one inference and its own instructions. How it is checked: in code, every line cites a waiver id or the status feed, the length is capped, and it is dated; then by the judge, on a sample, for groundedness against what it cites. Why it earns its place: the passages for one route do not say that the whole hub is closed; the briefing does, and it says it in every answer. Why it is dangerous: a wrong line is in every answer for five minutes. Its as-of is in the context, in the log, and in the answer's footer.

The dials

How often the briefing runs; how long a status may be old before the look up refuses it; how many candidates each index returns and how many the reranker keeps; the floor under which the assistant hands off; the context budget. Each trades freshness or recall for cost, and each is measured before it is set.

2. Data handling

The tables



The tables of the disruption assistant. Flights and bookings are read at the request and never indexed; a labelled question expires with the waiver it depends on.

Rules and waivers are kept by version, and a version is never edited: a changed waiver is a new row with a new version, and the old row's status changes. Chunks carry the source id and version, the kind, and for a waiver chunk the expiry, copied from the waiver so the check can read it without a join. Flights and bookings are other teams' tables, read by key and never indexed. Briefings are kept by version with the ids each one cites. The question log is one row per turn and holds the whole context with every as-of: the index version, the briefing version, the status minute, the quotes with their waiver ids and expiries, and the outcome. Outcomes join on the request id, minutes to weeks later. The labelled set is sampled from the log and pinned to the versions it depends on. Runs is one row per harness build.

A waiver as a row, exactly

COLUMN	VALUE
waiver id · version	W-2041 · v2
routes · dates	any flight through the hub · today and tomorrow
fare classes	all
what	rebook once without fee on any flight within seven days; a hotel tonight for passengers whose onward flight is tomorrow; no cash refund under this waiver
published · expires · withdrawn	18:05 · 23:59 tomorrow · (empty)
status	active

And its chunk: the waiver's text, one chunk, with the heading "Waiver W-2041 v2 · hub closure", the routes and dates in the text so the lexical index can find them, and `expires` copied into metadata so retrieval can filter on it and the check can read it. When v3 is published at 20:10 extending the hotel to two nights, the v2 row's status becomes superseded, v3 is a new row, and the index build inserts v3's chunk and deletes v2's in the same run.

The booking, and why it never enters the index

The booking is this passenger's and nobody else's: their flight, fare class, what they paid, what they have already been offered or claimed. It is read by key at the identify stage and placed in the context as lookups, with the minute it was read. It is never embedded, for two reasons that are both sufficient: it changes by the minute, so an indexed copy would be wrong; and the index is shared, so an indexed copy would be retrievable by the wrong passenger. The log holds it, hashed, because the log is what explains an answer later, and the log's readers are the team, not the passengers.

The labelled set, pinned

A labelled question holds the question, its slice (rule, status, both, unanswerable; and by kind: rebooking, refund, hotel, baggage), the chunk ids that answer it with their versions, the reference answer or NOT_FOUND, and the waiver versions it depends on. The rule for labellers is written: a passage is relevant if an agent would cite it; an answer is correct if an agent would give it, as of the waiver versions on the row. Two labellers, their agreement with its interval, adjudication, the rule amended. The row to slow down on is the last column: a question about the hotel waiver is a different question once the waiver is withdrawn. So a labelled row carries the versions it was true under, and the harness runs it against an index built from those versions, not from tonight's. When a waiver is withdrawn, its rows do not become wrong; they become a replay of a state the system was in, and a new row is labelled for the new state. The unanswerable slice is built the same way, and some of its rows are the answerable rows of a week ago whose waiver has gone: "is my hotel covered" is answerable at 18:12 and unanswerable at 21:35, and both are in the set.

One waiver, traced

TIME	WHAT HAPPENS	THE ANSWER THAT FOLLOWS
18:05	operations publishes W-2041 v1: rebook free, hotel tonight	
18:05	the waiver feed starts the index build; one chunk embedded, inserted; check; pointer	
18:08	the pointer moves to index v3,207; the cache keys on it, so every entry for this hub is stale	
18:10	the briefing job runs; the briefing now names W-2041	
18:12	a passenger asks "my flight was cancelled, what can I do?"	the first answer quoting W-2041: "rebook once without fee … a hotel tonight …" [2] W-2041 · expires 23:59 tomorrow; status for their flight as of 18:11; briefing as of 18:10
21:30	operations withdraws the hotel clause: W-2041 v2, hotel removed; v1 status withdrawn	
21:30	the index build runs on the change: v2's chunk inserted, v1's deleted, in one run; pointer at 21:33	
21:33	every cached answer for the hub is stale: new index version	
21:35	the briefing runs; the hotel line is gone	
21:35	a passenger asks the same question	the first answer that no longer quotes the hotel: rebooking from W-2041 v2 [2]; for the hotel, the floor: "the waiver no longer covers a hotel; here is who to ask"
21:31 to 21:33	the window: a question in these two minutes could still retrieve v1's chunk	the check reads the chunk's expiry and the waiver's status at the answer; a quote from a withdrawn version fails the check and is not shown. The window is measured and on the dashboard

The last row is the design's answer to its own freshness number: the index is minutes behind, so the check is the component that closes the gap, by re-reading the one fact that can change faster than the index, the waiver's status, from the waivers table at the moment of the answer.

3. Measurement

Before any passenger sees it

Per stage, on the labelled set, pinned to versions:

STAGE	MEASURED	AGAINST
the router	accuracy per label; the BOTH questions routed as RULE are the failure that matters	the labelled slice; three runs for stability
retrieval	recall at 6 per retriever and per slice; the waiver slice on its own, because waivers are short and full of route codes that only the lexical index finds	the relevant chunk ids, at their versions
the reranker	recall at 6 after reranking; the drop from 50 is the point	the same
the floor	of the unanswerable slice, how many are stopped; of the answerable, how many are stopped wrongly	the labelled set, both slices
generation	groundedness per claim by the judge; correctness against the reference by the judge; the quote check's pass rate in code	the judge validated against the team's labels: sensitivity, specificity, kappa, the corrected rate
the briefing	groundedness of every line against what it cites	its own labelled sample
the check	the share of drafts it rejects, by reason: a bad citation, a quote that is not a substring, an expired waiver	every answer; this one needs no labels
the whole	the report row per slice: rebooking, refund, hotel, baggage, unanswerable	the previous build

The unanswerable slice is the third of the set the prompt promised: compensation the rules do not give, a promise an agent made by phone, a hotel the airline does not offer tonight, and the questions that were answerable last week under a waiver that has gone. It measures two things: whether the floor stops the question before generation, and when it does not, whether the model says NOT_FOUND rather than writing a fluent answer from a passage that does not apply. The second number is the failure of the week.

With passengers

The primary metric is a rebooking completed in the session after an answer that offered one, per slice, against the previous version in an interleaved or split rollout. Behind it: the handoff rate, the thumbs, the refund claims decided against the answer the assistant gave, and the complaints that cite the assistant. A release is decided by the primary metric with its interval, and it is stopped by any guardrail.

The guardrail: **the quote check's rejection rate and the judge's groundedness rate on the money slices, refund and hotel**. A version that lifts rebookings by writing more confident answers will show up here first, because confident answers about money are exactly what the check and the judge are built to catch. A second guardrail is the count of answers whose quoted waiver was withdrawn before the answer was shown: the window on the trace table, which has to be near zero and is measured on every answer.

The pipeline's own measures

Freshness as a number: the time from a waiver's publish to the pointer, and from withdrawal to the pointer, per waiver, on the dashboard. The briefing's age at the moment of each answer. The status table's age per flight. The cache hit rate and the age of the entry that was served. The vendor's rate, the queue depth in front of it, and how many answers were served degraded. Each is a line in the log row, so each is a chart.

Bonus: the degraded mode

At 150 a second against 50 generations a second, two questions in three cannot be generated. The design serves them anyway, in this order. First, the cache: the same rewritten question on the same flight in the same minute is the same answer, and in a storm most questions are the same question. Second, for a cache miss when the vendor's queue is over its budget, the assistant skips generation and shows the search result: the flight's status and options from the table, and the active waiver quoted whole with its id and expiry, under a heading that says the assistant could not write a summary. Every fact the passenger sees is still a verbatim quote or a status read, so nothing is promised that the rules do not give. The log row records `degraded`, and the dashboard shows the share. What the system never does is widen the budget by skipping the check, shortening the passages below the waiver's condition, or answering from the briefing alone: a degraded answer is a smaller true answer, never a faster guess.

The one question: the hotel at 19:40

What the log holds. The row for that turn: the passenger, hashed; the flight key; the question and its rewrite; the index version and its as-of; the briefing version and its as-of; the status minute; the six passages with ids, versions and expiries, one of them the hotel clause of the waiver, version and expiry included; the context, whole; the draft with its quote and the waiver id; the check's result; the answer shown; the outcome, when it arrives. Everything needed to say what the system knew at 19:40.

Which component let it through. The index. The withdrawal at 19:30 reached the index build, but the pointer had not moved by 19:40: ten minutes against a freshness number of minutes means the build failed its check or the feed was late, and the dashboard's publish-to-pointer line says which. The retrieval stage did what it should with the index it had; the generate stage quoted a passage it was given.

Which component should have stopped it. The check. Its last rule reads the quoted waiver's status and expiry at the moment of the answer, from the waivers table, not from the chunk's copy. A quote from a version whose status is withdrawn fails the check, and the passenger sees the floor, not the promise. A design without that rule has its freshness guarantee only as strong as its slowest build. A design with it has a second guarantee that holds while the first is catching up, and the trace table shows the window it covers.